

# Gradient Boosting & XGBoost

A from-scratch learning guide covering boosting, XGBoost control and tuning, evaluation, and responsible model interpretation — built to prepare you to solve every Day 28 task independently.

Phase 2 · Week 6 · Day 28 · Data Science Learning Program

1. Today's Big Picture
12. Subsampling & Feature Sampling
2. Random Forest vs Boosting
13. XGBoost vs LightGBM vs CatBoost
3. Gradient Boosting From Scratch
14. Feature Importance
4. Residuals & Sequential Learning
15. Permutation Importance
5. Manual Boosting
16. Partial Dependence Plots
6. XGBoost Fundamentals
17. Responsible Model Interpretation
7. Important XGBoost Parameters
18. How to Solve the Day 28 Tasks
8. Evaluation & Overfitting
19. Common Mistakes
9. Early Stopping
20. Final Mental Model
10. Learning Rate vs Number of Trees
21. Quick Revision Cheat Sheet
11. Tree Depth & Overfitting
22. Practice Questions

# 1. Today's Big Picture

Yesterday you built a **Random Forest**: hundreds of trees, each grown independently on its own random slice of the data, all shouting their vote at the same time and the majority wins. Today you're building something structurally different — a team of trees that work **in relay**, where each runner only exists to fix the distance the previous runner fell short by.

That's the entire idea behind **gradient boosting**, and **XGBoost** is simply a very fast, very well-engineered library that implements it. You'll build the relay by hand first (so there's no mystery left), then hand the job to XGBoost, and spend most of the day learning to *control* it — because boosting is more powerful than bagging, and also much easier to break.

**One-line intuition:** Random Forest is a room full of independent experts voting at the same time. Boosting is one apprentice after another, each one only asked to fix what the previous apprentice still got wrong.

Everything today uses the same Titanic dataset, the same cleaning recipe, and the same `random_state=42` train/test split as yesterday — on purpose, so that when you compare Random Forest against XGBoost, any difference you see is a real difference between the algorithms, not an accident of a different split.

# Random Forest vs Boosting

The single most important distinction of the day.

| Random Forest (bagging)                               | Gradient Boosting                                                     |
|-------------------------------------------------------|-----------------------------------------------------------------------|
| Many trees trained <b>independently</b> , in parallel | Trees trained <b>sequentially</b> , one after another                 |
| Each tree sees a random bootstrap sample              | Each tree sees the <b>errors</b> left by all trees before it          |
| Final answer: trees <b>vote</b> / <b>average</b>      | Final answer: corrections are <b>summed</b> onto a running prediction |
| Mainly reduces <b>variance</b>                        | Mainly reduces <b>bias</b>                                            |
| Forgiving — defaults usually work                     | Powerful but sensitive — bad settings overfit fast                    |
| Trees are usually deep and complex                    | Trees are usually shallow (depth 3–6)                                 |

**Bagging vs Boosting in one line:** Bagging builds many independent trees and averages away their random errors. Boosting builds trees one after another, where each tree's job is to correct the running total's remaining error.

Why does bagging reduce variance and boosting reduce bias? A single deep tree overfits (high variance) — averaging many independent overfit trees cancels out their individual noise, which is what a Random Forest does. A single shallow tree underfits (high bias) — it's too simple to capture the pattern. Boosting fixes that not by making one tree bigger, but by adding more shallow trees that each patch up the leftover error, which chips away at bias round after round.

## SECTION 3

# Gradient Boosting From Scratch

Building the intuition before any formula.

### WHAT IS IT?

Gradient boosting is a way to build one strong prediction model out of many weak ones, by training them one at a time and always aiming the next tree at whatever the current team is still getting wrong.

### WHY DO WE NEED IT?

A single small tree is a weak learner — it captures only a rough pattern. Instead of building one huge tree that tries to capture everything (and overfits), boosting builds many small trees, each contributing a small, targeted correction. The team ends up far more accurate than any single member.

### HOW DOES IT WORK?

**Real-world analogy:** Imagine grading an essay in rounds. Round 1: a first grader gives a rough score. Round 2: a second grader looks specifically at what the first grader missed, and adjusts the score up or down. Round 3: a third grader looks at what's *still* off after rounds 1 and 2, and nudges again. After enough rounds, the accumulated score is far better than any single grader's guess — and each grader only had to be right about a small slice of the mistake.

Concretely, gradient boosting for regression (and the classification version works the same way under the hood) follows this loop:

1. Start with a dumb baseline prediction — usually just the average of the target.
2. Compute the **residual** for every row:  $\text{residual} = \text{actual} - \text{predicted}$ .
3. Train a small tree whose only job is to predict those residuals.
4. Add a **fraction** of that tree's prediction to the running total (the fraction is the  $\text{learning\_rate}$ ).
5. Recompute residuals against the new, slightly-improved prediction.
6. Repeat for many rounds.

**Simple example:** Predicting Titanic survival (0 or 1). Baseline = mean survival rate, say 0.38, for every passenger. A passenger who survived (actual = 1) has residual =  $1 - 0.38 = 0.62$  — a positive number, meaning "push the prediction up." A passenger who died (actual = 0) has residual =  $0 - 0.38 = -0.38$  — a negative number, meaning "push the prediction down." Every round, the next tree learns which passengers still need pushing, and in which direction.

### PYTHON IDEA

```
prediction = np.full(len(y_train), y_train.mean()) # baseline
for round_num in range(n_rounds):
    residual = y_train.values - prediction # what's still wrong
    tree = DecisionTreeRegressor(max_depth=2).fit(X_train, residual)
    prediction += learning_rate * tree.predict(X_train) # small nudge
```

### RESULT INTERPRETATION

Plot the training error against the round number and it should fall steadily — never jumping around — because each new tree is fit directly to whatever error is currently left, so it can only help, never hurt, the training score (though it can absolutely hurt the test score if you run too many rounds — more on that in Section 9).

**Common mistake:** Confusing gradient boosting with a Random Forest that happens to run trees "in a loop." The defining feature isn't the loop — it's that every tree is trained on the error of the trees before it, not on independent random data.

# Residuals & Sequential Learning

The engine that makes boosting work.

**Residual** — the gap between the truth and the current prediction, for each row:  $\text{residual} = \text{actual} - \text{predicted}$  . It is literally the error that is still left over, so a tree trained on residuals is a tree trained to fix what is still broken.

**Why is the sign of the residual meaningful?** The sign tells the next tree which direction to push:

- Positive residual (actual > predicted) → the model is under-predicting this row → push the prediction up.
- Negative residual (actual < predicted) → the model is over-predicting this row → push the prediction down.
- Residual near zero → this row is already well explained → leave it alone.

**Why does fitting a tree to residuals = fitting a tree to the remaining error?** Because the residual *is*, by definition, everything the current prediction has not yet explained. A tree that predicts the residual well is a tree that has found a pattern in what's still wrong — and adding its prediction (scaled by the learning rate) directly closes that gap.

## SEQUENTIAL VS INDEPENDENT, SIDE BY SIDE

|                             | Random Forest tree                             | Boosting tree                              |
|-----------------------------|------------------------------------------------|--------------------------------------------|
| Trained on                  | a random bootstrap sample of the original data | the residuals left by every tree before it |
| Knows about other trees?    | No — built independently                       | Yes — it only exists because of them       |
| Can be trained in parallel? | Yes                                            | No — order matters                         |

## Manual Boosting

Building the loop with your own hands, before XGBoost touches anything.

This section directly prepares you for **Task 01** and **Task 02**. The whole point of building boosting by hand is that once you've done it, XGBoost stops looking like a black box — it's the same loop, just heavily optimized.

### WHY MAX\_DEPTH=2 ON THE LITTLE TREES?

Each tree in the loop is deliberately kept tiny (2 levels deep = at most 4 leaves). It doesn't need to be smart on its own — it only has to explain a small slice of the remaining error. A weak learner is the whole point: many weak, cheap correctors stacked together beat one expensive, complicated model, and they're far less likely to memorize noise.

### WHY DOES LEARNING\_RATE CONTROL HOW MUCH EACH TREE IS ALLOWED TO CHANGE THE PREDICTION?

Look at the update line: `prediction += learning_rate * tree.predict(X_train)`. The tree computes a full correction, but only a *fraction* of it (e.g. 30% if `learning_rate=0.3`) actually gets applied. This is a deliberate act of caution — if you applied the whole correction every round, the very next residual would overcorrect in the opposite direction, and the model would oscillate and chase noise instead of converging smoothly.

**Analogy:** This is exactly the same caution as the learning rate in gradient descent (Day 12). Taking small, repeated steps toward the fix, rather than one giant leap, is what makes the process stable.

### WHY DOES TRAINING ERROR FALL ALMOST EVERY ROUND?

Because each tree is fit directly against the current residual, it always finds *some* pattern to exploit in the training data (even if that pattern is just noise, given enough rounds). That's why training error keeps dropping steadily — but as you'll see in Section 8, a falling training error is not the same thing as a model that's getting better.

### FULL WORKED LOOP (WHAT TASK 02 ASKS YOU TO REPRODUCE)

```
prediction = np.full(len(y_train), y_train.mean())
trees = []
for round_num in range(30):
    residual = y_train.values - prediction
    tree = DecisionTreeRegressor(max_depth=2).fit(X_train, residual)
    prediction += learning_rate * tree.predict(X_train)
    trees.append(tree)

# to predict on new data, replay the same recipe from the same baseline:
def predict(X, trees, baseline, learning_rate):
    pred = np.full(len(X), baseline)
    for tree in trees:
        pred += learning_rate * tree.predict(X)
    return pred
```

**Common mistake:** Forgetting to start the *test-set* prediction from the same baseline mean used on the training set, or forgetting to apply every stored tree in the same order they were trained. The predict function must replay the exact recipe, not just the last tree.

**What happens with learning\_rate = 1.0?** You'd apply the whole correction every round instead of a fraction. Early rounds will drop error fast, but the model will start chasing individual noisy rows and the error curve will look jagged or even bounce back up, instead of smoothly flattening. This is exactly what Task 02 asks you to observe.

## XGBoost Fundamentals

From the hand-built loop to a production-grade library.

**XGBoost** — short for eXtreme Gradient Boosting — a highly optimized implementation of the exact loop you just built by hand, with added regularization, clever handling of missing values, and serious speed engineering.

### WHY IS XGBOOST A BOOSTING ALGORITHM?

Under the hood it does the same thing: start from a baseline, compute how wrong the current model is, fit a new (shallow) tree to correct that error, scale the correction by a learning rate, and repeat. What it adds on top is engineering, not a new idea — smarter tree-building, built-in regularization terms that discourage overly complex trees, native handling of missing values, and heavy optimization for speed.

### WHY IS IT POWERFUL, AND WHY CAN IT OVERFIT?

Because it can chase the training residual with great precision, it will happily keep improving the training score forever if you let it — which means, unchecked, it can memorize the training set. Section 8 and 9 are entirely about preventing that.

### MINIMAL USAGE

```
import xgboost as xgb
model = xgb.XGBClassifier(
    n_estimators=300, learning_rate=0.1, max_depth=3,
    eval_metric="logloss", random_state=42
).fit(X_train, y_train)

proba = model.predict_proba(X_test)[: , 1]  # predicted probability of class 1
```

On the Titanic dataset, with matched settings, XGBoost edges past the Random Forest from yesterday — the gap is modest on a dataset this small (about 1,000 rows), but on larger structured/tabular datasets gradient boosting usually wins by a clear margin, which is why it dominates competitions and production systems for table-shaped data.

Random Forest is forgiving — its defaults usually work fine out of the box. XGBoost is more powerful but also more sensitive: get the hyperparameters wrong and it overfits enthusiastically. The rest of this guide is about controlling that properly.

# Important XGBoost Parameters

The knobs that matter for today's tasks.

| Parameter             | Meaning                                                | ↑ Increase                                       | ↓ Decrease                                     |
|-----------------------|--------------------------------------------------------|--------------------------------------------------|------------------------------------------------|
| n_estimators          | Number of boosting rounds (trees)                      | More capacity, more overfit risk                 | Less capacity, may underfit                    |
| learning_rate         | Fraction of each tree's correction applied             | Faster convergence, riskier                      | Slower, more trees needed, usually more stable |
| max_depth             | How many questions deep each tree may go               | More complex trees, more overfit risk            | Simpler trees, may underfit                    |
| eval_metric           | Metric used to score each eval round (e.g. "logloss" ) | Not a strength dial — just picks what's measured |                                                |
| random_state          | Seed for reproducibility                               | Not a strength dial — controls randomness only   |                                                |
| early_stopping_rounds | Stop if no eval improvement for this many rounds       | More patience before stopping                    | Stops sooner, safer against overfit            |
| eval_set              | Data watched each round to decide when to stop         | Required for early stopping to work              |                                                |
| subsample             | Fraction of training rows each tree sees               | Less randomness, closer to full data             | More randomness, can reduce overfitting        |
| colsample_bytree      | Fraction of features each tree may use                 | Less randomness                                  | More randomness, can reduce overfitting        |

**Common mistake:** Treating `eval_metric` or `random_state` as tuning knobs. They don't make the model stronger or weaker — they control what gets measured and how reproducible the randomness is.



## Evaluation & Overfitting

Knowing whether the model is actually good.

### WHAT IS ROC-AUC, INTUITIVELY?

Imagine picking one random passenger who survived and one random passenger who didn't. ROC-AUC is the probability your model ranks the survivor's predicted probability higher than the non-survivor's. A score of 1.0 means perfect ranking every time; 0.5 means the model is no better than a coin flip; below 0.5 means it's ranking things backwards.

ROC-AUC is a *ranking* metric, not a threshold metric — it doesn't care whether you predicted "survived" at 0.5 or 0.51 cutoff, only whether survivors tend to score higher than non-survivors overall. That makes it a fair way to compare models without committing to a decision threshold.

### TRAINING SCORE VS TEST SCORE

- **Training ROC-AUC** — how well the model ranks the data it was trained on. This can always be pushed toward 1.0 by adding more trees or depth — it is not proof of a good model.
- **Test ROC-AUC** — how well the model ranks data it has never seen. This is the number that matters for real-world use.
- **Train-test gap** — the difference between the two. A small gap means the model generalizes well. A large, widening gap is the clearest overfitting warning sign you will get.

Why a high training score alone is not enough: a model that has simply memorized the training passengers will score close to a perfect 1.0 on training data while doing no better than chance on new passengers. The training score tells you almost nothing about real-world performance by itself.

**Why must you always use the same train/test split when comparing models?** Because if you re-split the data for a different model, any score difference you measure could just be luck from that particular split — easy rows landing in one model's test set and hard rows in another's — rather than a genuine difference in model quality. Reusing the same `random_state=42` split removes that doubt, which is exactly why the Task Sheet insists on it for the Random Forest vs. XGBoost comparison.

## SECTION 9

# Early Stopping

Letting the model tell you when to stop adding trees.

**Early stopping** — train with a validation set watched at every round; if the validation score has not improved for a set number of rounds ( `early_stopping_rounds` ), stop training and keep the best round. Prevents overfitting and saves time without any manual tuning.

### WHY DOES ASKING FOR TOO MANY TREES BECOME SAFE?

Instead of guessing the right `n_estimators` , you can deliberately ask for far more trees than needed (e.g. 1000) and let early stopping cut training off automatically once validation performance stops improving. On the Titanic example, asking for 1000 trees with `early_stopping_rounds=30` actually stopped far short of 1000 — and the test score went **up**, not down, compared to a fixed 300-tree model. Those extra hundreds of trees would only have fit noise.

### TRAINING SET, VALIDATION SET, TEST SET

| Set            | Used for                                                                        |
|----------------|---------------------------------------------------------------------------------|
| Training set   | Fitting the trees — what the model learns from directly                         |
| Validation set | Watched during training to decide <i>when to stop</i> — never used to fit trees |
| Test set       | Touched only at the very end, to report a final, honest score                   |

**Important note from the Task Sheet:** Using the test set as the `eval_set` is acceptable for this teaching exercise, but it is not good real-world practice — the model becomes indirectly influenced by the test data through the stopping decision. In a real project you would carve a separate validation set out of the training data instead. Write one sentence in your notebook acknowledging this, as the Task Sheet asks.

### THE SHAPE OF THE TWO CURVES

Plot training logloss and validation logloss against boosting round on the same axes:

- Training loss keeps falling, essentially forever — a boosted model can always fit its own training data a little better by adding one more tree.
- Validation loss falls, flattens, and then starts creeping **upward**. That upturn is overfitting beginning.
- `best_iteration` marks the round right before that upturn — the point early stopping locks in.

This is one of the most important pictures in applied machine learning: the same shape you saw with the bias-variance curve back on Day 23, appearing again in a completely different model — which tells you it's a property of learning itself, not a quirk of one algorithm.

```
model_es = xgb.XGBClassifier(
    n_estimators=1000, learning_rate=0.1, max_depth=3,
    eval_metric="logloss", early_stopping_rounds=30, random_state=42
)
model_es.fit(X_train, y_train,
            eval_set=[(X_train, y_train), (X_test, y_test)], verbose=False)

print(model_es.best_iteration)          # round it actually stopped at
results = model_es.evals_result()       # dict of per-round metrics
```

## SECTION 10

# Learning Rate vs Number of Trees

Two knobs that pull against each other.

**Analogy:** Getting to a destination in large steps (fewer steps needed) vs. small steps (many more steps needed). Both can arrive at roughly the same place — but large steps risk overshooting and stumbling near the target, while small steps are safer but need far more of them.

A small `learning_rate` means each tree contributes a tiny correction, so you need many more trees to accumulate the same total fix. A large `learning_rate` arrives in fewer trees but risks overshooting past the right answer. On the Titanic example, four very differently-scaled pairs — `(0.3, 50)`, `(0.1, 150)`, `(0.05, 300)`, `(0.01, 1500)` — land at broadly similar test scores, because each pairing delivers roughly the same total amount of correction.

Neither setting is "automatically better" — **small learning rate  $\neq$  automatically better**, and **large learning rate  $\neq$  automatically better**. They must be considered together as one trade-off, not tuned independently.

## THE PRACTICAL RULE

Pick a smallish, sensible learning rate by judgement (commonly 0.05–0.1), set `n_estimators` generously high, and let **early stopping** decide the actual tree count. That way you never have to hand-tune the number of trees at all — Task 05 asks you to state exactly this rule.

**Common mistake:** Pairing a tiny learning rate with too few trees (e.g. `lr=0.01, n=50`) — the model never accumulates enough correction and underperforms. Or pairing a large learning rate with far too many trees (e.g. `lr=0.3, n=1500`) — early rounds converge fast, but the extra trees keep chasing noise with large steps, risking instability and overfitting.

# Tree Depth & Overfitting

The single most direct overfitting dial in a boosted model.

**max\_depth** — how many questions deep each individual tree may go. In boosting we keep trees deliberately shallow (often 3 to 6), because each tree only needs to fix a small piece of the remaining error — not solve the whole problem alone.

| Depth | Train AUC                        | Test AUC                    | Gap            |
|-------|----------------------------------|-----------------------------|----------------|
| 2     | rises modestly                   | competitive                 | small (~0.01)  |
| 3     | rises further                    | often the best or near-best | small-moderate |
| 5     | rises sharply                    | starts to fall              | widening       |
| 8     | approaches ~0.999 (near-perfect) | falls clearly               | large (>0.20)  |

(Exact numbers vary by run — the shape of the pattern is what matters: as depth grows, training AUC climbs toward a near-perfect score while test AUC falls, and the gap between them widens from roughly 0.01 at depth 2 to over 0.20 at depth 8.)

As trees get deeper, training AUC climbs toward a near-perfect score — the model is essentially memorizing the training passengers. Meanwhile test AUC falls. The widening gap is the clearest warning sign you will get that the model has stopped learning general patterns and started learning the training set itself.

## WHY DOES RANDOM FOREST USE DEEP TREES SUCCESSFULLY, WHILE BOOSTING PREFERS SHALLOW ONES?

In a Random Forest, each deep tree is allowed to overfit its own bootstrap sample individually — but because the trees are independent and see different random samples, averaging hundreds of them cancels out each tree's individual overfitting (that's the variance-reduction mechanism from Section 2). In boosting there's no such averaging-away step: every tree's output is added directly onto a shared running prediction. If any one tree is deep and overfits, that overfit signal gets baked permanently into the final model rather than averaged away. So boosting relies on each individual tree staying weak and simple, letting the sequence of many small corrections do the work instead.

## SECTION 12

# Subsampling & Feature Sampling

Borrowing randomness from the Random Forest.

**subsample** — the fraction of training ROWS each tree sees (e.g. 0.8 = 80% of passengers, chosen at random per tree).

**colsample\_bytree** — the fraction of FEATURES each tree may use (e.g. 0.8 = 80% of the columns, chosen at random per tree). Directly analogous to **max\_features** in a Random Forest.

Row sampling and feature sampling both inject controlled randomness so that no single tree can lean too heavily on one dominant feature or one particular group of rows — which can reduce overfitting, the same mechanism that makes bagging work.

On a small dataset like the 1,000-row Titanic sample, this randomness often doesn't help much — there simply isn't enough data for subsampling to add useful variety, and holding data back can even cost a little accuracy. On larger datasets, values around 0.8 are a commonly recommended default and tend to help more.

**The honest lesson:** Do not apply a "recommended" setting like **subsample=0.8** blindly because a tutorial said so. Test it on your actual data — and check across a couple of different **random\_state** values, because on a small dataset a single seed can make randomness look helpful or harmful purely by chance.

# XGBoost vs LightGBM vs CatBoost

Three libraries, the same core idea.

**LightGBM** — Microsoft's implementation, built for speed. Grows trees leaf-by-leaf rather than level-by-level, which makes it noticeably faster on large datasets — often the right pick when you have millions of rows.

**CatBoost** — Yandex's implementation, built to handle categorical columns natively. You can hand it text categories directly without one-hot encoding them first, avoiding the leakage risks of naive target encoding.

| Library  | Main strength                 | Reach for it when...                             |
|----------|-------------------------------|--------------------------------------------------|
| XGBoost  | Reliable, well-tested default | Starting point for most tabular problems         |
| LightGBM | Speed on large data           | Training time becomes painful (millions of rows) |
| CatBoost | Native categorical handling   | Many high-cardinality categorical columns        |

All three implement essentially the same underlying gradient boosting algorithm, so on a small, all-numeric dataset like this one they land within a whisker of each other on accuracy — that's the honest and expected result, not a sign that one library is smarter. Speed differences between them only show up clearly at much larger row counts than 1,000, and CatBoost's advantage only shows up when you actually have categorical columns to hand it directly — this dataset's features are all numeric, so that advantage can't appear here.

```
m_lgb = lgb.LGBMClassifier(n_estimators=200, learning_rate=0.1, max_depth=3, random_state=42)
m_cat = CatBoostClassifier(iterations=200, learning_rate=0.1, depth=3, random_state=42, verbose=0)
```

**Common mistake:** Forgetting that CatBoost uses different parameter names — `iterations` instead of `n_estimators` , and `depth` instead of `max_depth` .

## Feature Importance

Reading a boosted model — the built-in way, and why to be careful with it.

**Impurity-based (built-in) importance** — `model.feature_importances_` scores each feature by how much it reduced impurity (error) across all the splits that used it, summed over every tree.

**Why can it mislead?** This method is biased toward features with many possible split points — continuous numeric features (like `fare` or `age`) get more chances to be picked for a split than a binary feature, purely because they offer more places to cut. This is the same **high-cardinality bias** you proved on Day 27 with the Random Forest: a feature that's genuinely useless (pure random noise) but has many unique values can still rack up a non-trivial impurity score, purely from being split on often, not because it actually helps predictions.

# Permutation Importance

The more trustworthy alternative.

**Permutation importance** — shuffle one feature's values (breaking its relationship with the target) and measure how much the model's score drops. A feature that genuinely matters causes a big drop when shuffled; a useless feature causes almost none.

```
from sklearn.inspection import permutation_importance
perm = permutation_importance(model, X_test, y_test, n_repeats=20,
                             random_state=42, scoring="roc_auc")
```

Because permutation importance is measured directly against real predictive performance (on the test set) rather than against how often a feature was used for splitting, it isn't fooled by a feature simply having lots of unique values. This is why the Task Sheet has you compare both methods side by side and check where a deliberately-added pure-noise column ranks under each — the noise column should rank low under permutation importance, but can rank unfairly high under impurity-based importance.

|                 | Impurity-based (built-in)                    | Permutation importance                             |
|-----------------|----------------------------------------------|----------------------------------------------------|
| Measures        | How often/effectively a feature was split on | Actual drop in real predictive score when shuffled |
| Biased toward   | High-cardinality / continuous features       | Not biased this way                                |
| Cost            | Free (already computed during training)      | More expensive (re-scores many times)              |
| Trustworthiness | Use with caution                             | Generally preferred                                |



## Partial Dependence Plots (PDP)

Seeing how the model's prediction changes as one feature moves.

**Partial Dependence Plot** — holds every other feature fixed and sweeps one feature across its range, plotting the model's *average* predicted outcome at each value. It shows the average relationship the model has learned between that one feature and the prediction.

### HOW TO READ ONE

- **Jump** — a sudden change in predicted outcome around a specific feature value (e.g. survival probability jumping sharply between `pclass` 1 and `pclass` 3).
- **Curve** — a smooth, gradual relationship (e.g. survival probability easing down as `age` increases).
- **Plateau** — a flat region where changing the feature barely moves the prediction, meaning the model isn't sensitive to that feature in that range.

PDP shows the model's average learned relationship — it is **not causal evidence**. A PDP showing that higher fare is associated with higher predicted survival does not mean paying more money causes survival; it means the model has learned that fare correlates with something that matters (likely passenger class and cabin location).

**Two caveats to keep in mind:** (1) When features are correlated with each other (e.g. `fare` and `pclass`), a PDP can imply feature combinations that don't actually exist in real data — sweeping `fare` alone while holding `pclass` fixed can create an unrealistic "expensive third-class ticket" scenario. (2) The plot always shows an *average* effect across all passengers — it can hide the fact that the relationship works differently for different subgroups.

## Responsible Model Interpretation

The distinction that matters most.

A feature being **useful for prediction** does NOT automatically mean the feature **causes** the outcome.

`sex_male` dominates both Random Forest and XGBoost's importance rankings on Titanic — being male is strongly *predictive* of not surviving, because of the "women and children first" evacuation policy. Being male didn't *cause* a lower survival chance in some direct physical sense — it correlated with being deprioritized during evacuation. Both model families agreeing on `sex_male`, `pclass`, and `fare` mattering is a genuinely good sign that the pattern is real and not an artefact of one particular algorithm — but "real pattern the model can exploit" and "causal mechanism" are two different claims, and only the first one is something these models can actually support.

**If two model families trained completely differently agree** on which features matter, that agreement is reassuring evidence the pattern is a genuine property of the data, not a quirk of one algorithm's biases. **If they disagree sharply** on a feature, that's a signal to investigate before trusting either ranking — check for high correlation with another feature, look at the PDP shape for that feature in both models, and consider whether one model's importance method (impurity-based vs. permutation) is the more trustworthy read [here](#).

# How to Solve the Day 28 Tasks

Task-by-task reasoning — not finished answers.

## Task 01 — Residuals by hand

|              |                                                                                                                              |
|--------------|------------------------------------------------------------------------------------------------------------------------------|
| Asking       | Compute a baseline prediction (mean of <code>y_train</code> ) and the residual for each row.                                 |
| Tests        | Whether you understand a residual as "the error still left over."                                                            |
| Think first  | What does a positive vs. negative residual mean about the direction the prediction needs to move? (See Section 4.)           |
| Method       | <code>prediction = np.full(len(y_train), y_train.mean())</code> , then <code>residual = y_train.values - prediction</code> . |
| Key function | <code>np.mean((y_train.values - prediction)**2)</code> for MSE.                                                              |
| Expect       | Every prediction is identical (the mean); residuals split cleanly around it based on <code>actual</code> .                   |
| Interpret    | Positive residual → push up; negative → push down; near zero → leave alone.                                                  |
| Avoid        | Computing residuals against anything other than the flat baseline mean at this stage.                                        |

## Task 02 — Manual gradient boosting loop

|              |                                                                                                                                                  |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Asking       | Implement the full boosting loop yourself over 30 rounds, without any boosting library.                                                          |
| Tests        | Whether you can reproduce the mechanics from Section 5 unaided.                                                                                  |
| Think first  | Why does error fall nearly every round, and what would <code>learning_rate=1.0</code> do differently?                                            |
| Method       | Baseline → loop: residual → fit <code>DecisionTreeRegressor(max_depth=2)</code> → update prediction → store tree.                                |
| Key function | <code>DecisionTreeRegressor(max_depth=2).fit(X_train, residual)</code> .                                                                         |
| Expect       | A smoothly falling training-MSE curve across 30 rounds.                                                                                          |
| Interpret    | A falling curve confirms each tree is successfully explaining leftover error.                                                                    |
| Avoid        | Writing a separate predict path that doesn't replay the exact same baseline + tree sequence used in training (see the mistake box in Section 5). |

## Task 03 — Bagging vs boosting, side by side

|              |                                                                                                                          |
|--------------|--------------------------------------------------------------------------------------------------------------------------|
| Asking       | Compare Random Forest and XGBoost with matched settings, same split, reporting both train and test ROC-AUC plus the gap. |
| Tests        | Whether you can fairly evaluate overfitting risk, not just accuracy.                                                     |
| Think first  | A higher test score with a much larger train-test gap is a warning, even if it's currently ahead — why?                  |
| Method       | Fit both models on the identical <code>X_train, y_train</code> ; score both on train and test.                           |
| Key function | <code>roc_auc_score(y_train, model.predict_proba(X_train)[: ,1])</code> in addition to the test score.                   |
| Expect       | XGBoost likely edges ahead on test AUC but may show a larger train-test gap than Random Forest.                          |
| Interpret    | A larger gap means the lead could erode on future unseen data — it's a fragility signal, not just a footnote.            |
| Avoid        | Re-splitting the data for one of the two models — must be the identical split (Section 8).                               |

## Task 04 — Early stopping

|              |                                                                                                                                                                          |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Asking       | Fit with <code>n_estimators=1000</code> , <code>early_stopping_rounds=30</code> , plot train vs. validation logloss with <code>best_iteration</code> marked.             |
| Tests        | Understanding of Section 9 — why stopping early can raise, not lower, the score.                                                                                         |
| Think first  | Why isn't "the score went up when we stopped early" a contradiction?                                                                                                     |
| Method       | <code>fit(..., eval_set=[(X_train,y_train),(X_test,y_test)], verbose=False)</code> , then read <code>model.best_iteration</code> and <code>model.evals_result()</code> . |
| Key function | <code>evals_result()</code> , <code>ax.axvline(model.best_iteration)</code> .                                                                                            |

|                  |                                                                                                                                         |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Expect</b>    | Stopping far short of 1000 rounds; test AUC at or above the fixed 300-tree model from Task 03.                                          |
| <b>Interpret</b> | Training loss keeps falling after the stop point; validation loss would have risen — the extra trees would have only fit noise.         |
| <b>Avoid</b>     | Forgetting the one-sentence note that using the test set as <code>eval_set</code> is acceptable here but not ideal real-world practice. |

#### Task 05 — Learning rate vs tree count

|                    |                                                                                                                                                              |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Asking</b>      | Compare four matched (lr, n) pairs, then two deliberately mismatched pairs.                                                                                  |
| <b>Tests</b>       | The trade-off from Section 10.                                                                                                                               |
| <b>Think first</b> | Why would <code>lr=0.01, n=50</code> underperform, and what risk does <code>lr=0.3, n=1500</code> carry?                                                     |
| <b>Method</b>      | Loop over the pairs, fit, score test ROC-AUC each time.                                                                                                      |
| <b>Expect</b>      | The four matched pairs land close together; the mismatched pair with too few trees at a tiny learning rate clearly underperforms.                            |
| <b>Interpret</b>   | Too-few-trees-at-tiny-lr never accumulates enough correction (underfits); too-many-trees-at-large-lr risks instability/overfitting.                          |
| <b>Avoid</b>       | Concluding one single learning rate is "best" instead of stating the practical rule: fix learning rate by judgement, let early stopping pick the tree count. |

#### Task 06 — Max depth and overfitting

|                    |                                                                                                                      |
|--------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Asking</b>      | Sweep <code>max_depth</code> over 2, 3, 5, 8; report train AUC, test AUC, and the gap for each.                      |
| <b>Tests</b>       | Section 11's core lesson.                                                                                            |
| <b>Think first</b> | Why does Random Forest tolerate deep trees when boosting doesn't? (Averaging vs. direct summation — see Section 11.) |
| <b>Method</b>      | Loop over depths, fit with fixed <code>n_estimators=200, learning_rate=0.1</code> , score train and test.            |
| <b>Expect</b>      | Train AUC climbs toward ~1.0; test AUC peaks at a shallow depth then falls; gap widens sharply by depth 8.           |
| <b>Interpret</b>   | Name the depth with the best test AUC as the answer — not the depth with the best training AUC.                      |
| <b>Avoid</b>       | Picking the depth with the highest training score as "best."                                                         |

#### Task 07 — Randomness settings, tested honestly

|                    |                                                                                                                                      |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Asking</b>      | Compare <code>subsample</code> / <code>colsample_bytree</code> at (1.0,1.0), (0.8,0.8), (0.6,0.6), then repeat with different seeds. |
| <b>Tests</b>       | Whether you'll trust a default blindly or check it against evidence (Section 12).                                                    |
| <b>Think first</b> | Would you expect a small dataset to show a clear benefit from subsampling?                                                           |
| <b>Method</b>      | Loop the three (ss, cs) pairs; repeat with <code>random_state=7</code> and <code>123</code> to check stability.                      |
| <b>Expect</b>      | Little or no consistent benefit on ~1,000 rows; results may even flip direction across seeds.                                        |
| <b>Interpret</b>   | State plainly whether randomness helped, hurt, or was noise — and whether the conclusion held across seeds.                          |
| <b>Avoid</b>       | Applying the commonly-cited 0.8 default without testing it on this specific dataset first.                                           |

#### Task 08 — XGBoost vs LightGBM vs CatBoost

|                     |                                                                                                                       |
|---------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Asking</b>       | Matched-settings comparison across all three libraries, timed, averaged over 3 runs.                                  |
| <b>Tests</b>        | Section 13 — knowing where these libraries actually differ.                                                           |
| <b>Think first</b>  | On which axis (rows, categorical columns) would differences become obvious, and why can't this dataset show it?       |
| <b>Method</b>       | Time each <code>.fit()</code> call with <code>time.time()</code> , repeat 3× per library, average.                    |
| <b>Key function</b> | CatBoost uses <code>iterations</code> / <code>depth</code> , not <code>n_estimators</code> / <code>max_depth</code> . |
| <b>Expect</b>       | Near-identical accuracy across all three; timing differences too small on 1,000 rows to be meaningful.                |
| <b>Interpret</b>    | Write a short, honest decision rule — don't claim an accuracy winner the data can't support.                          |

|              |                                                                        |
|--------------|------------------------------------------------------------------------|
| <b>Avoid</b> | Declaring one library "more accurate" from a single small-dataset run. |
|--------------|------------------------------------------------------------------------|

### Task 09 — Importance methods and the noise feature

|                    |                                                                                                                                                                       |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Asking</b>      | Compare built-in vs. permutation importance on the Task 04 model, including a synthetic noise column.                                                                 |
| <b>Tests</b>       | Section 14–15's high-cardinality bias lesson, applied to a new model.                                                                                                 |
| <b>Think first</b> | Where would you expect a meaningless noise column to rank under each method?                                                                                          |
| <b>Method</b>      | <code>model.feature_importances_</code> vs. <code>permutation_importance(..., scoring="roc_auc")</code> ; add <code>np.random.normal</code> column, retrain, recheck. |
| <b>Expect</b>      | Noise column ranks near the bottom under permutation importance; may rank surprisingly high under impurity-based importance.                                          |
| <b>Interpret</b>   | Explain in two sentences why continuous noise can inflate impurity-based scores (many split points, not real signal).                                                 |
| <b>Avoid</b>       | Trusting <code>feature_importances_</code> as the final word without the permutation cross-check.                                                                     |

### Task 10 — Comparing what the two models learned

|                    |                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Asking</b>      | Side-by-side permutation importance and PDPs for Random Forest and XGBoost on the same test set.          |
| <b>Tests</b>       | Section 17 — cross-model agreement as evidence of a real pattern.                                         |
| <b>Think first</b> | What would agreement between the two models suggest? What would disagreement demand you investigate?      |
| <b>Method</b>      | Run <code>permutation_importance</code> on both fitted models; build a PDP for the top feature from each. |
| <b>Expect</b>      | Broad agreement on <code>sex_male</code> , <code>pclass</code> , <code>fare</code> as top features.       |
| <b>Interpret</b>   | State which findings are robust across model families vs. specific to one algorithm.                      |
| <b>Avoid</b>       | Treating any single model's ranking as ground truth without the cross-check.                              |

### Task 11 — Model selection memo

|                    |                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Asking</b>      | A one-page Markdown memo recommending Random Forest or XGBoost for production, with numbers, a risk, and a next step.                        |
| <b>Tests</b>       | Whether you can synthesize everything into a decision a non-technical reader can act on.                                                     |
| <b>Think first</b> | What's your single strongest reason — and what's the honest risk of your pick?                                                               |
| <b>Method</b>      | Pull the Task 03 numbers (test AUC, train-test gap for both), the Task 04 hyperparameters/best_iteration, and the Task 10 feature agreement. |
| <b>Expect</b>      | A short memo with a clear recommendation, exact hyperparameters, one named risk, one next test.                                              |
| <b>Interpret</b>   | Never claim a model is "better" without naming the metric and the margin.                                                                    |
| <b>Avoid</b>       | Vague claims like "XGBoost is stronger" without a specific AUC number and gap to back it up.                                                 |

# Common Mistakes

The traps that catch people on Day 28.

| Mistake                                                                 | Why it's a problem                                                                                              |
|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Judging a model by training score alone                                 | Training score can always be pushed toward perfect by adding trees/depth — it says nothing about generalization |
| Re-splitting data between model comparisons                             | Any difference measured could be due to the split, not the algorithm                                            |
| Applying <code>subsample=0.8</code> or similar defaults without testing | What helps on large datasets may do nothing (or hurt) on small ones                                             |
| Trusting <code>feature_importances_</code> alone                        | Biased toward high-cardinality/continuous features                                                              |
| Treating "predictive" as "causal"                                       | A feature can be useful for prediction purely through correlation, not causation                                |
| Using deep trees in a boosted model                                     | Unlike Random Forest, boosting has no averaging step to cancel out one tree's overfitting                       |
| Forgetting CatBoost's different parameter names                         | <code>iterations</code> / <code>depth</code> , not <code>n_estimators</code> / <code>max_depth</code>           |
| Reading a single small-dataset timing run as a real speed conclusion    | Library speed differences only show up at much larger scale                                                     |

## SECTION 20

# Final Mental Model

Everything, connected in one page.

### Random Forest

Many independent trees →  
vote together →  
reduces **variance**.

### Gradient Boosting

One tree after another →  
each fixes previous errors →  
reduces **bias** → powerful, but easier to overfit.

**XGBoost** = controlled, optimized gradient boosting, tuned through: learning rate · tree depth · number of trees · early stopping · row/feature sampling · honest evaluation.

Model → Evaluation → Tuning → Interpretation → Responsible Decision

## SECTION 21

# Quick Revision Cheat Sheet

Compact code reference.

```
XGBClassifier(...)
```

Creates an XGBoost classifier with chosen hyperparameters.

```
DecisionTreeRegressor(max_depth=2)
```

The weak-learner tree used both by hand and inside boosting libraries.

```
.fit(X, y)
```

Trains the model on features and target.

```
.predict(X)
```

Returns hard class predictions.

```
.predict_proba(X)[:, 1]
```

Returns predicted probability of the positive class.

```
eval_set=[(X_tr,y_tr),(X_te,y_te)]
```

Data watched each round for early stopping / logging.

```
.evals_result()
```

Dict of per-round metric history for every eval set.

```
.best_iteration
```

Round number early stopping actually settled on.

```
.feature_importances_
```

Built-in, impurity-based importance — use with caution.

```
permutation_importance(...)
```

Trustworthy importance via shuffling and re-scoring.

```
PartialDependenceDisplay
```

sklearn tool for plotting a feature's average marginal effect.

```
roc_auc_score(y_true, proba)
```

Ranking-quality metric independent of decision threshold.



## Practice Questions

Test yourself before opening the notebook.

**Q1.** A passenger has actual = 0 and the current model predicts 0.65 for them. What is the residual, and what does its sign tell the next tree to do?

**Q2.** Why does adding more trees always help the *training* score in boosting, but not necessarily the *test* score?

**Q3.** You have two candidate settings: `lr=0.2, n=100` and `lr=0.02, n=1000`. Both reach similar test AUC. Which practical advantage does the second one give up, and which risk does the first one carry?

**Q4.** A colleague trains a boosted model with `max_depth=10` and reports 99.8% training accuracy. Should they be pleased? What single number would you ask for next?

**Q5.** Why can't a Random Forest's success with deep trees be copied directly into a boosting model?

**Q6.** A synthetic noise column ranks 2nd out of 7 features under `feature_importances_` but last under permutation importance. Which ranking do you trust, and why?

**Q7.** Two different model families (Random Forest and XGBoost) both rank the same feature as most important. What can you reasonably conclude — and what can you *not* conclude — from that agreement?

**Q8.** You are asked to justify using the test set as `eval_set` for early stopping in a course exercise. What is the honest limitation of doing this in a real production project?